EPFC

PRO
Programmation orientée objet

Leçon 5

Wrappers

Génériques

Listes

L'interface `Comparable<E>`

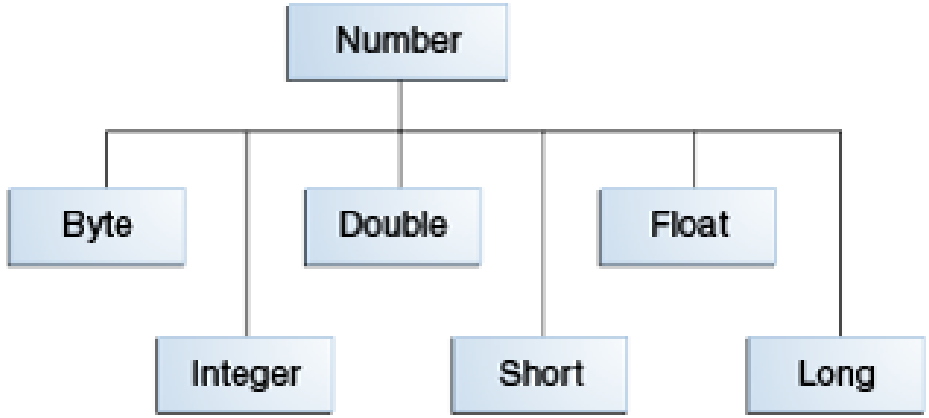
EPFC

Les classes d'emballage (*wrappers*)

Les classes d'emballage (*wrappers*)

Java fournit des classes d'emballage (*wrappers*) pour chacun des types primitifs.

Ces classes « emballent » les primitifs dans un objet.



(… et d'autres)

Wrappers : avantages et inconvénients

Les *wrappers* sont utiles quand une méthode attend un objet (`equals, compareTo`, ...).

Ils **uniformisent le code** en supprimant les différences entre les types primitifs et types références.

En plus, ces classes fournissent des méthodes et des constantes utiles :

```
Integer.MAX_VALUE;
Integer.valueOf("123");
...
```

Par contre, les *wrappers* sont **moins efficaces** pour un usage intensif.

Wrappers : boxing et *unboxing*

EPFC

```
Integer x = new Integer(2);
Integer y = 12;              //boxing
int z = x;                   //unboxing de x
System.out.println(y + z); //unboxing de y
```

Le compilateur fait certaines conversions lui même :

Boxing :

convertir un primitif vers un objet

Unboxing :

convertir un objet vers un primitif.

Comparaison : ==, equals et compareTo

```
Integer a = new Integer(41);
Integer b = new Integer(42);
Integer c = new Integer(42);
int     d = 42;

if (b == c)
    System.out.println("Ce message n'est jamais affiché");

if (b.equals(c))
    System.out.println("b identique à c");

if (a.compareTo(b) < 0)
    System.out.println("a plus petit que b");

if (b == d)
    System.out.println("b == d"); //unboxing de b
```

La généricité

Méthodes génériques

Les méthodes génériques ont un paramètre de type (E dans l'exemple) indiqué entre chevrons avant le type de retour de la méthode.

Exemple :

```
public static <E> void reverse(E[] arr) {
    for(int k = 0; k < arr.length / 2; ++k) {
        E tmp = arr[k];
        arr[k] = arr[arr.length - 1 - k];
        arr[arr.length - 1 - k] = tmp;
    }
}
```

```
Integer[] a1 = {1, 2, 3};
reverse(a1);

Person[] a2 = {new Person("a"), ...};
reverse(a2);

int[] a3 = {1, 2, 3};
reverse(a3);
```

Le paramètre de type doit être un type référence, pas un primitif.

Classes génériques

Une classe générique a un (ou plusieurs) paramètre(s) de type placé(s) entre chevrons juste après son nom.

```
public class Pair<E> {
    private E first, second;

    public Pair(E first, E second) {
        this.first = first;
        this.second = second;
    }

    public E getFirst() {
        return this.first;
    }

    public E getSecond() {
        return this.second;
    }
}
```

```
Pair<Integer> p = new Pair<>(2, 3);
System.out.println(p.getFirst());
```

EPFC

Les listes de la librairie Java

`java.util.List<E>`

EPFC

Une liste (`List<E>`) est une collection ordonnée d'éléments du même type.

Comme toutes les collections Java, leur taille est dynamique.

Une `ArrayList<E>` est une liste implémentée à l'aide d'un tableau.
Une `LinkedList<E>` est implémentée à l'aide d'une liste liée.

On peut parcourir une liste à l'aide de la boucle foreach.

Méthodes courantes de `List<E>`

Méthode	Comportement
`E get(int index)`	Renvoie l'élément situé à la position donnée.
`E set(int index, E elem)`	Remplace l'élément situé à la position donnée.
`boolean add(E elem)`	Ajoute l'élément donné à la fin de la liste.
`void add(int index, E elem)`	Ajoute l'élément donné à la position spécifiée.
`E remove(int index)`	Supprime et renvoie l'élément situé à la position donnée.
`int size()`	Renvoie le nombre d'éléments.

Attention à ne pas confondre `set` et `add`.

Exemple d'utilisation de `List<E>`

```
Person person1 = new Person("Jean");

List<Person> list = new ArrayList<>();

list.add(person1);
list.add(new Person("Paul"));

for(Person p : list)
    System.out.println(p);

for(int i = 0; i < list.size(); ++i)
    System.out.println(list.get(i));
```

L'interface `Comparable<E>`

L'interface Comparable<E>

L'interface Comparable<E> permet d'indiquer à la librairie Java que les instances d'une classe sont comparables à des objets de type E et que donc la méthode compareTo(E o) y est implémentée.

```
public class Person implements Comparable<Person> {
    //...
    @Override
    public int compareTo(Person p) {
        return -this.dateOfBirth.compareTo(p.dateOfBirth);
    }
}
```

La méthode statique Collections.sort(List<E>) trie une liste d'éléments comparables (qui implémentent donc l'interface Comparable<E>)